



Secrets Management Activation Day

Rob Whitelocks

Snr Security Specialist
Solutions Architect

Alice Wanjohi

Solutions Architect

Geroen Joris

Snr Security Specialist
Solutions Architect

Welcome



- Slides shared at the end
- Recording for internal training & quality
- We encourage interactions and questions
- Question about Topic use Q&A Panel
- Technical issues or general chat use Chat Panel
- Please provide Feedback
- Other Activation Days available:
<https://aws-experience.com/emea/smb/events/series/activation-days>

Agenda

AWS Secrets Manager Overview

Secret Lifecycle Management

Workshop

Recap

Real world Attacks - Customer incidents

- AWS CIRT engagements
- Top 2 initial access techniques



Public-facing application compromise

Compromised credentials

**“If you want to keep a
secret, you must also
hide it from yourself”**

George Orwell

1984



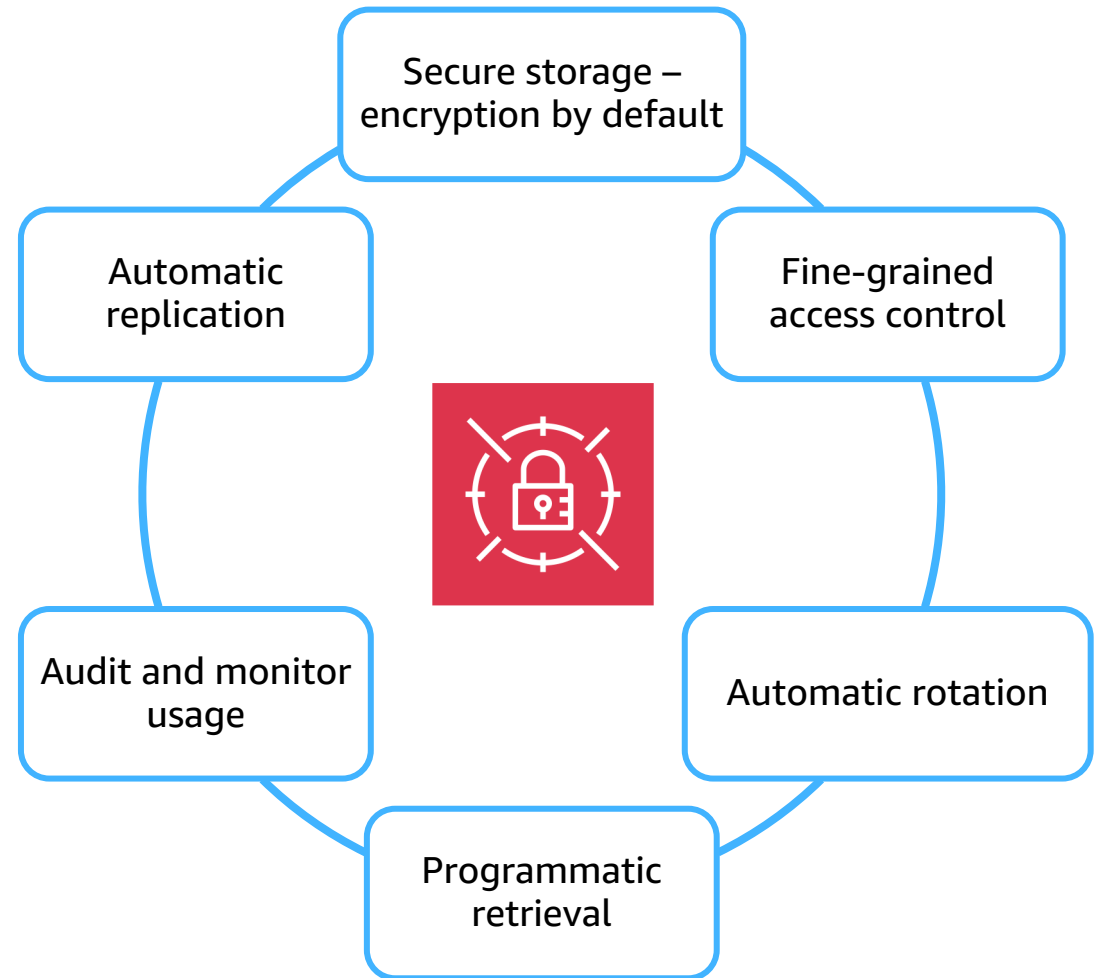
AWS Secrets Manager Overview

What is AWS Secrets Manager?

AWS Secrets Manager is a fully managed service that helps customers **manage, retrieve, and rotate** database credentials, API keys, and other secrets throughout their lifecycles

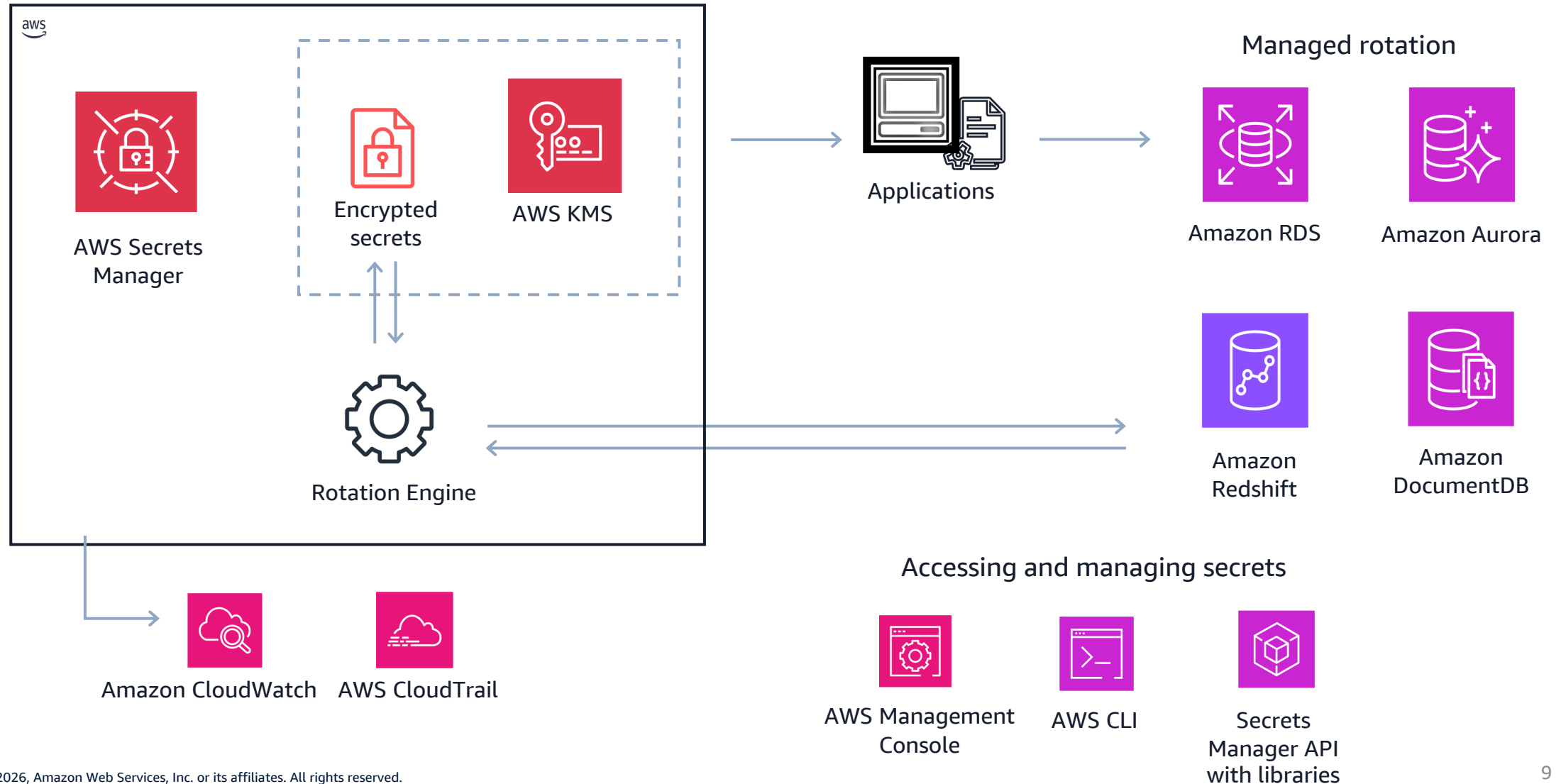
Why do customers use it?

1. Eliminate third-party licensing fees or self-managed infrastructure costs
2. Reduce operational overhead and secrets sprawl
3. Improve security posture and compliance



How Secrets Manager works

[AWS Secrets Manager](#)
[Rotation Lambda](#)
[Functions](#)



AWS Secrets Manager is a Regional service with **high availability**

North America

N. Virginia
Ohio
N. California
Oregon
Central Canada
Calgary
Central Mexico
GovCloud

South America

Sao Paulo

Africa

Cape Town

Europe

Frankfurt
Ireland
London
Milan
Paris
Spain
Stockholm
Zurich
Sovereign Cloud

Middle East

Tel Aviv
Bahrain
UAE

Asia Pacific

Hong Kong
Hyderabad
Jakarta
Malaysia
Melbourne
Mumbai
Osaka
Seoul
Singapore

Taipei
Thailand
Tokyo

Australia & New Zealand

New Zealand
Sydney

Example: rotation for Salesforce OAuth client credentials

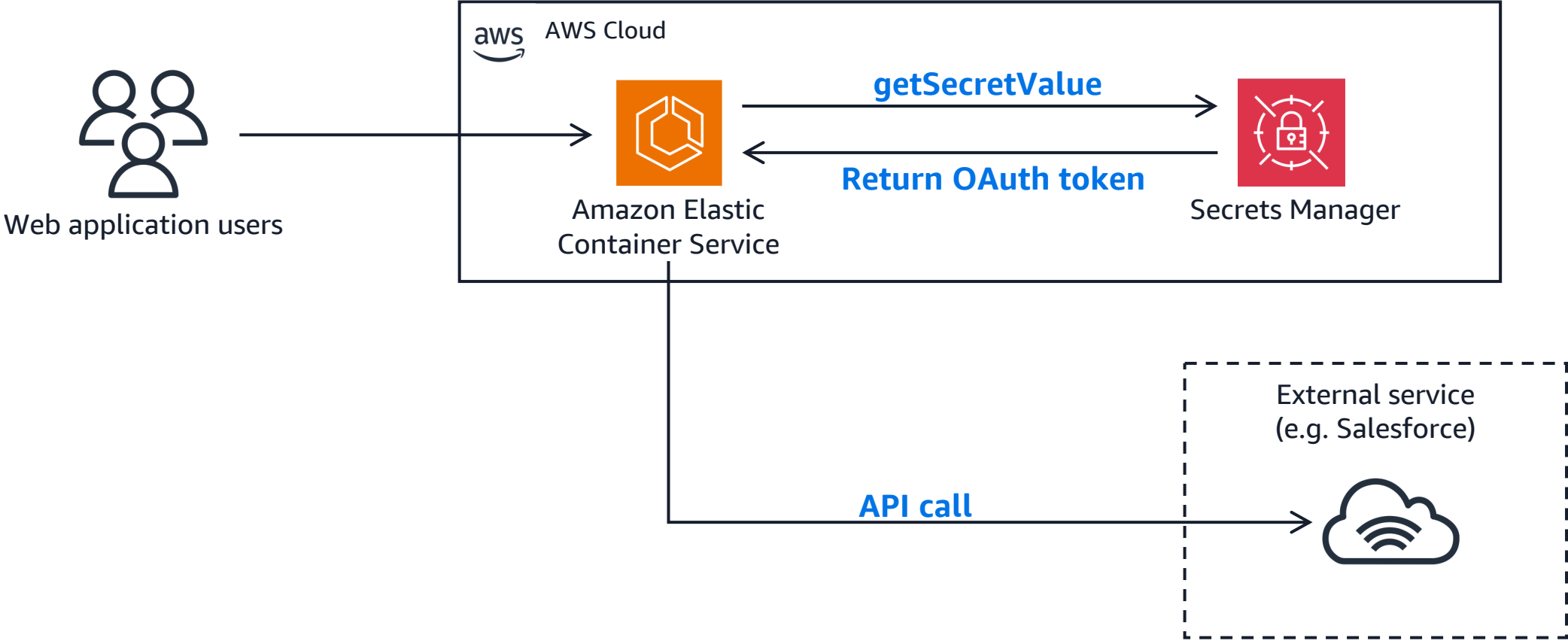
OAuth 2.0 client credentials grant is used to authenticate server-to-server API calls without user interaction. The application exchanges credentials for an access token, which authorizes subsequent API requests.

OAuth client credentials consists of 2 fields:

1. Client ID – identifies your connected app (public, non-sensitive)
2. Client secret – shared secret used to prove your app's identity to the authorization server (sensitive, must be rotated)

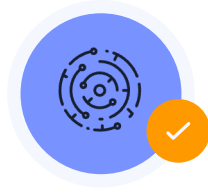
Secrets Manager stores the client secret and uses managed rotation to generate a new secret in Salesforce automatically, with zero application downtime

Salesforce OAuth client credentials example

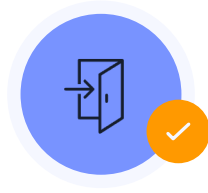




Benefits



Client secret is never stored locally with the application on disk



Developer is less likely to commit the client secret to source code

Client secret is accessed via the [Secrets Manager API](#)



If the client secret changes, then the value in Secrets Manager is changed

Will not require a redeployment of the application if that is required for configuration changes

With AWS Secrets Manager, you pay only for what you use, there is **no minimum fee**

zero

set-up fees or
commitments

New AWS customers will receive up to
\$200 in AWS Free Tier credits

\$0.40

Per secret per
month

\$0.05

Per 10,000 API calls

You are charged for number of
secrets you store and for API
requests made to the service
each month



AWS Secrets Manager's Key Differentiators

Client-side caching and getting secrets

Client-side caching of frequently used secrets provides several benefits:

1. **Reduced costs** – For secrets you use often, retrieving them from the cache reduces the number of API calls your applications make to Secrets Manager, inherently reducing cost
 2. **Improved performance** – Retrieving the secret from memory instead of sending a request over the Internet to Secrets Manager can dramatically improve the performance of applications
 3. **Improved availability** – Network errors are less of a problem when retrieving secrets from a cache
-

Supported Secrets Manager caching components:

1. [Client-side libraries](#) in Java, Python, .NET, Go, and Rust
2. [AWS Parameters and Secrets Lambda Extension](#) for Lambda
3. [AWS Secrets and Configuration Provider \(ASCP\)](#) for the K8s CSI Driver for EKS
4. [AWS Secrets Manager Agent](#) to standardize consumption across compute services



Cost optimization in Secrets Manager

Pricing

PER SECRET PER MONTH

\$0.40 per secret per month. A replica secret is considered a distinct secret and will also be billed at \$0.40 per replica per month. For secrets that are stored for less than a month, the price is prorated (based on the number of hours.)

If your organization plans to use a large volume of secrets, [contact us](#).

PER 10,000 API CALLS

\$0.05 per 10,000 API calls.

2. Clean up **old** and **unused** secrets using [AWS Config](#)

1. Utilize **client-side caching** or the **BatchGetSecretValue** API to reduce the number of API calls

Improve application availability and reduce costs with the **AWS Secrets Manager Agent**



[How to use Secrets Manager Agent blog](#)

A language-agnostic HTTP caching service that retrieves and caches secrets locally across any AWS compute environment

BEFORE

- ✗ Repeated API calls increase costs and reduce availability
- ✗ Multiple languages require different secret retrieval approaches
- ✗ Secret rotation forces container restarts and downtime
- ✗ Inconsistent access patterns across compute environments



AFTER

- ✓ Improves performance and cuts costs
- ✓ Standardized consumption across EC2, ECS, EKS, Lambda
- ✓ Language-agnostic HTTP interface works everywhere
- ✓ Zero-downtime secret rotation
- ✓ Built-in IAM controls and SSRF protection



Simplify secrets management in Amazon EKS with the **AWS Secrets Store EKS add-on**

A managed EKS plugin that mounts, caches, and rotates secrets with zero DevOps overhead



[How to use the EKS add-on with AWS Secrets Manager blog](#)

BEFORE

- ✗ Manual configuration of Helm charts, YAML, and ongoing maintenance
- ✗ No fine-grained access control at pod level
- ✗ Container restarts required during rotation
- ✗ Application unviability during regional disruption
- ✗ Scaling across hundreds of thousands of pods is unmanageable



AFTER

- ✓ One-click install: managed EKS add-on eliminates all manual configuration
- ✓ Fine-grained permissions at pod level with EKS Pod Identity
- ✓ Secrets mounted as files, rotated without restarting containers
- ✓ Automatic Region failover ensures high availability
- ✓ Secrets Manager at scale across entire Kubernetes clusters



AWS Secrets Manager is not just for AWS secrets

Simplify third-party credential management with **managed external secrets**

A new secret type that stores and automatically rotates third-party credentials



[Managed External Secrets for third-party credentials blog](#)

BEFORE

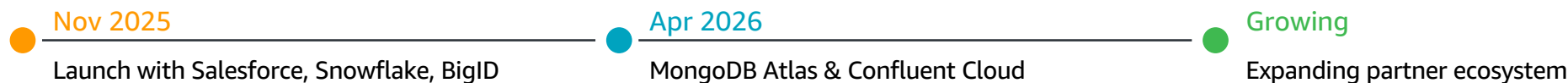
- ✗ Custom Lambda rotation functions per provider
- ✗ Manual metadata management
- ✗ Inconsistent security posture
- ✗ Operational overhead for every SaaS integration
- ✗ Different credential formats per vendor



AFTER

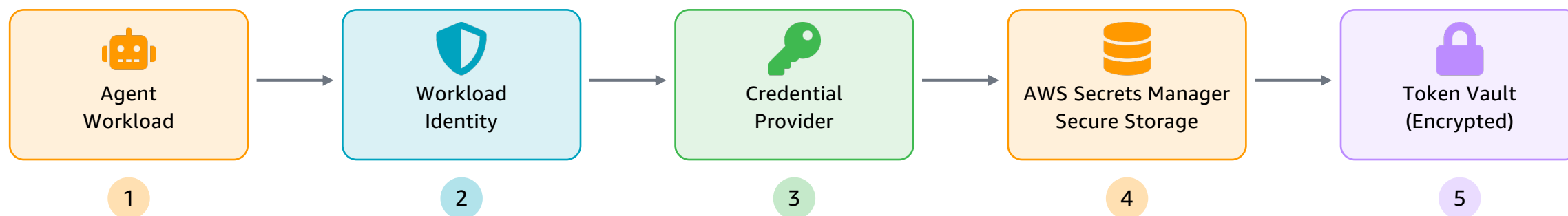
- ✓ Rotation ON by default — compliance is automatic
- ✓ Zero operational overhead, lambda free managed rotation
- ✓ Standardized secret format with metadata
- ✓ Same API for all providers
- ✓ Growing partner ecosystem

LAUNCH TIMELINE



Amazon Bedrock AgentCore Identity leverages Secrets Manager

AWS Secrets Manager is the secure storage backbone of AgentCore Identity's Credential Provider Pattern



- ✓ Zero Trust Enforcement
- ✓ CloudTrail audit logging

- ✓ Auto-provisioned secrets in Secrets Manager
- ✓ Agent credentials fully managed by service

Protect secrets from quantum threats with post-quantum TLS support enabled by default



[Protecting your secrets from tomorrow's quantum risks](#)

Protection against future threats

Defends secrets today from "harvest now, decrypt later" quantum attacks

Security of the cloud benefit

AWS-managed protection enabled automatically

Zero effort security

Automatically protect secrets in transit with hybrid PQ-TLS (ML-KEM), requiring no code changes or configuration for customers on the latest client versions

Engineered for zero performance impact

Optimized implementation ensures negligible effect on customer workloads

Global availability

ML-KEM post-quantum encryption deployed across all Regions (except China)



AWS Secrets Manager is PQC resistant by default.

How do we know?



[Protecting your secrets
from tomorrow's
quantum risks](#)



AWS Secrets
Manager

Verifying PQ-TLS usage in AWS Secrets Manager

- Look up CloudTrail logs for specified API calls
- keyExchange field will contain 'MLKEM' to indicate successful PQ connection

```
],
  "eventType": "AwsApiCall",
  "managementEvent": true,
  "recipientAccountId": "111122223333",
  "eventCategory": "Management",
  "tlsDetails": {
    "tlsVersion": "TLSv1.3",
    "cipherSuite": "TLS_AES_128_GCM_SHA256",
    "clientProvidedHostHeader": "secretsmanager.eu-west-2.amazonaws.com",
    "keyExchange": "X25519MLKEM768"
  }
}
```

AWS Secrets Manager integrates with **partner solutions**

 1Password



[Link](#)

 HashiCorp



[Link](#)



CYBERARK®
(Palo Alto Networks Idira)



[Link](#)

 AKEYLESS



[Link](#)



Deployment Considerations

To centralize or not to centralize? That is the question...

[Exploring common centralized and decentralized approaches to secrets management](#)



Creation

Considerations:

- Permissions
- KMS Key
- Tagging

Management

Considerations:

- Rotation
- Replication
- Versions

Consumption

Considerations:

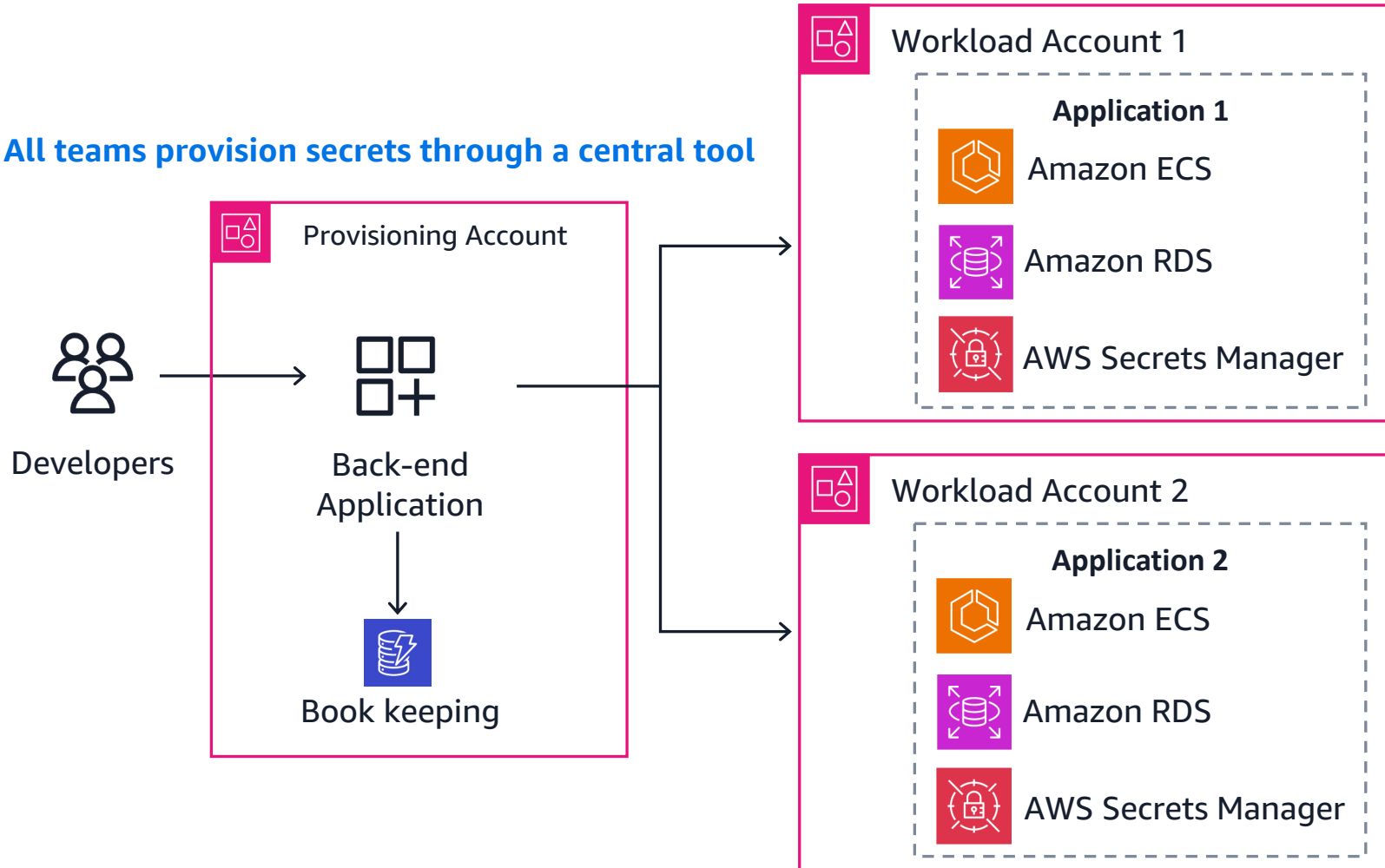
- Environment
- Caching
- CLI/ SDK

Observability/ Auditability/ Governance/ Compliance

It depends!

Centralized creation of secrets

All teams provision secrets through a central tool



Advantages:

- Consistent naming, tagging, and access control can be validated centrally
- Central repo for book-keeping any additional logs

Trade-offs:

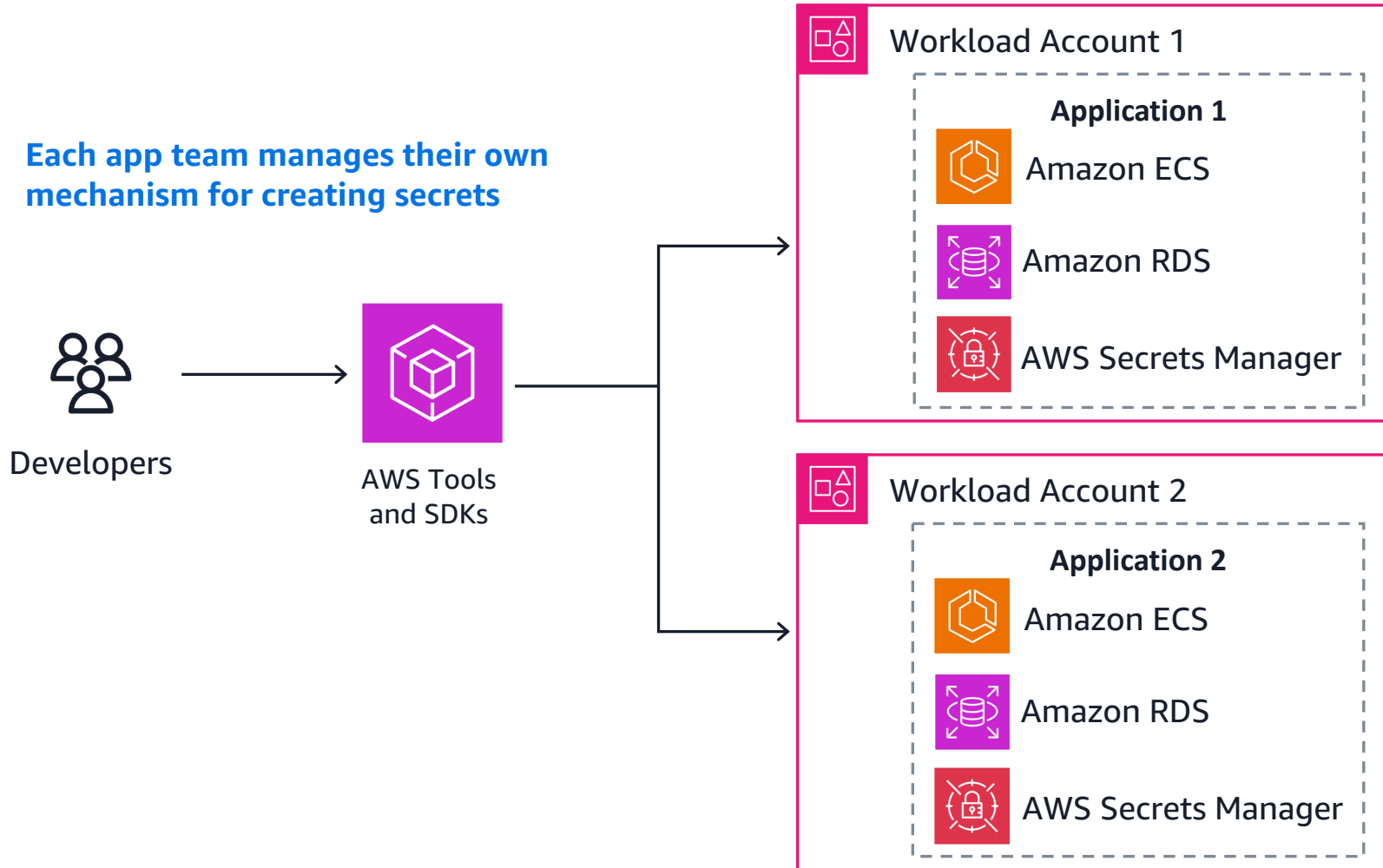
- Requires large initial overhead to set up provisioning account
- Overhead to include future releases

Bottom line:

- Centralized control is valuable for organizations requiring strict governance

De-centralized creation of secrets

Each app team manages their own mechanism for creating secrets



Advantages:

- Simpler to adopt (no initial setup needed)
- Developers have full flexibility and control

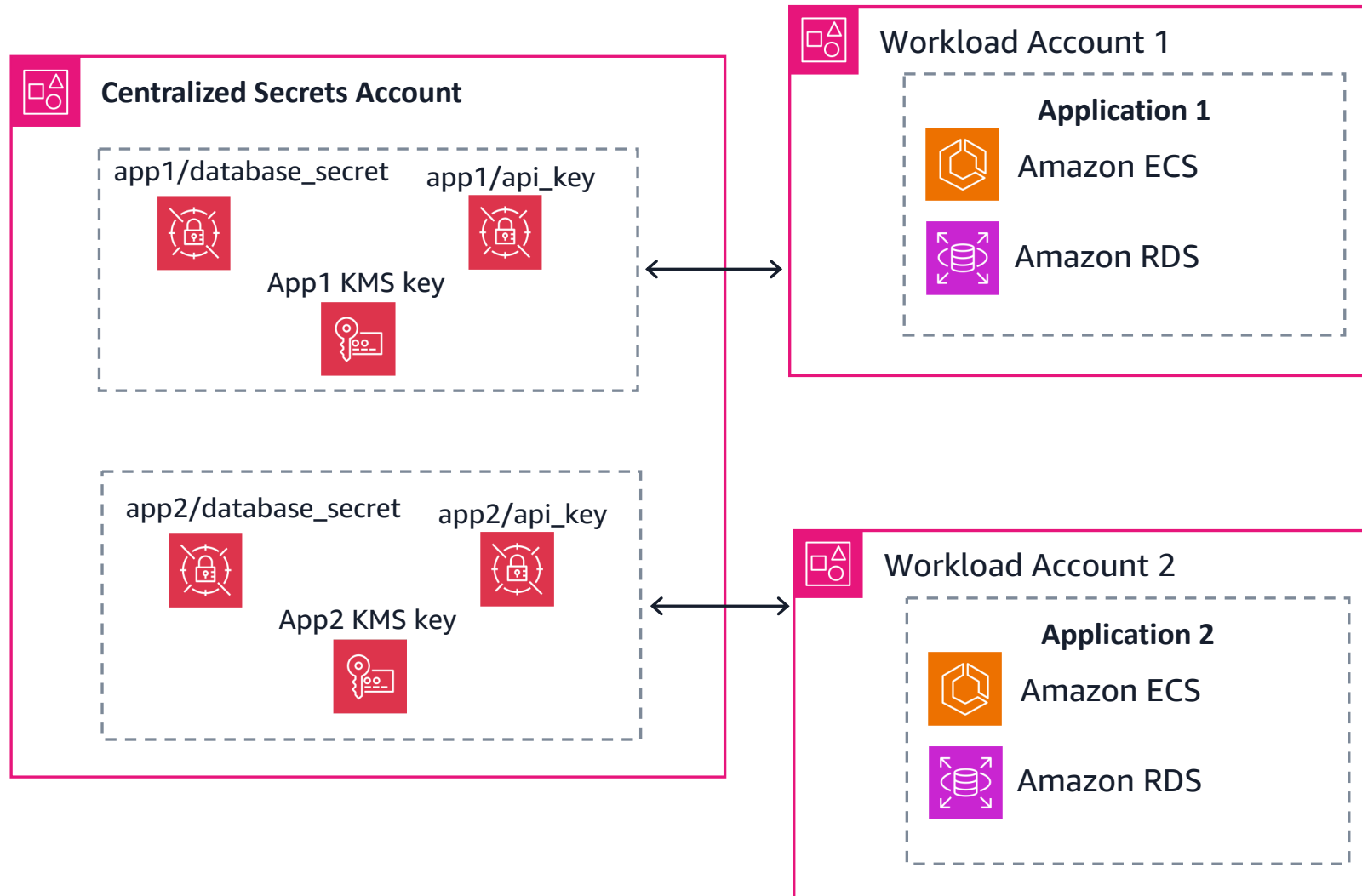
Trade-offs:

- Requires additional efforts to implement org level name, tags and access controls
- Logs are separated into different accounts

Bottom Line:

- More common model
- It gives developers autonomy while still implementing controls through guardrails and centralized logging

Centralized consumption of secrets



Advantages:

- Full control over all secrets of the organization from a single account
- Centralized logs for audit and compliance

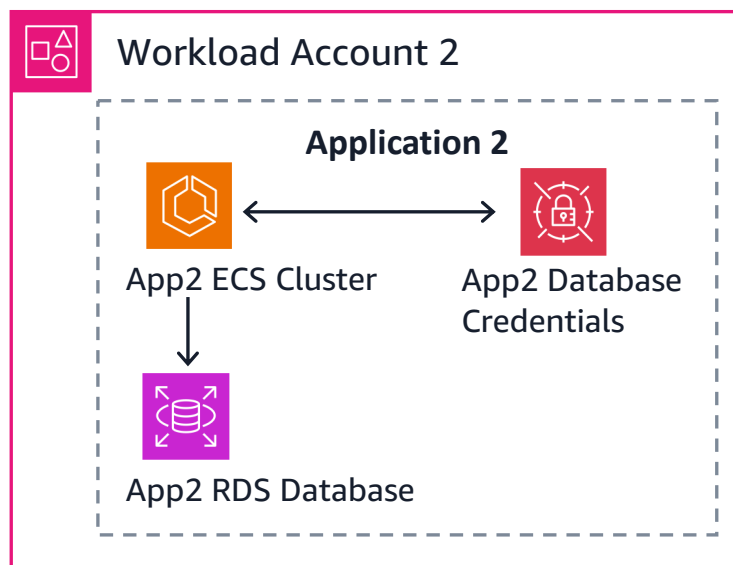
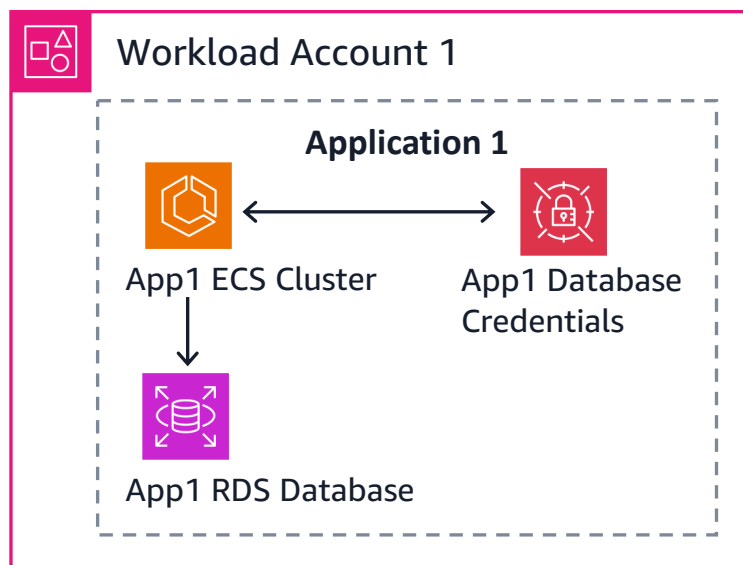
Trade-offs:

- Single account holds secrets of entire org
- Features such as Managed secrets are not supported

Bottom Line:

- Gives you control and audit visibility but creates impact radius risk and blocks managed rotation features

De-centralized consumption of secrets



Advantages:

- Secrets are separated into logical segment for each team
- Permissions management delegated to app owners

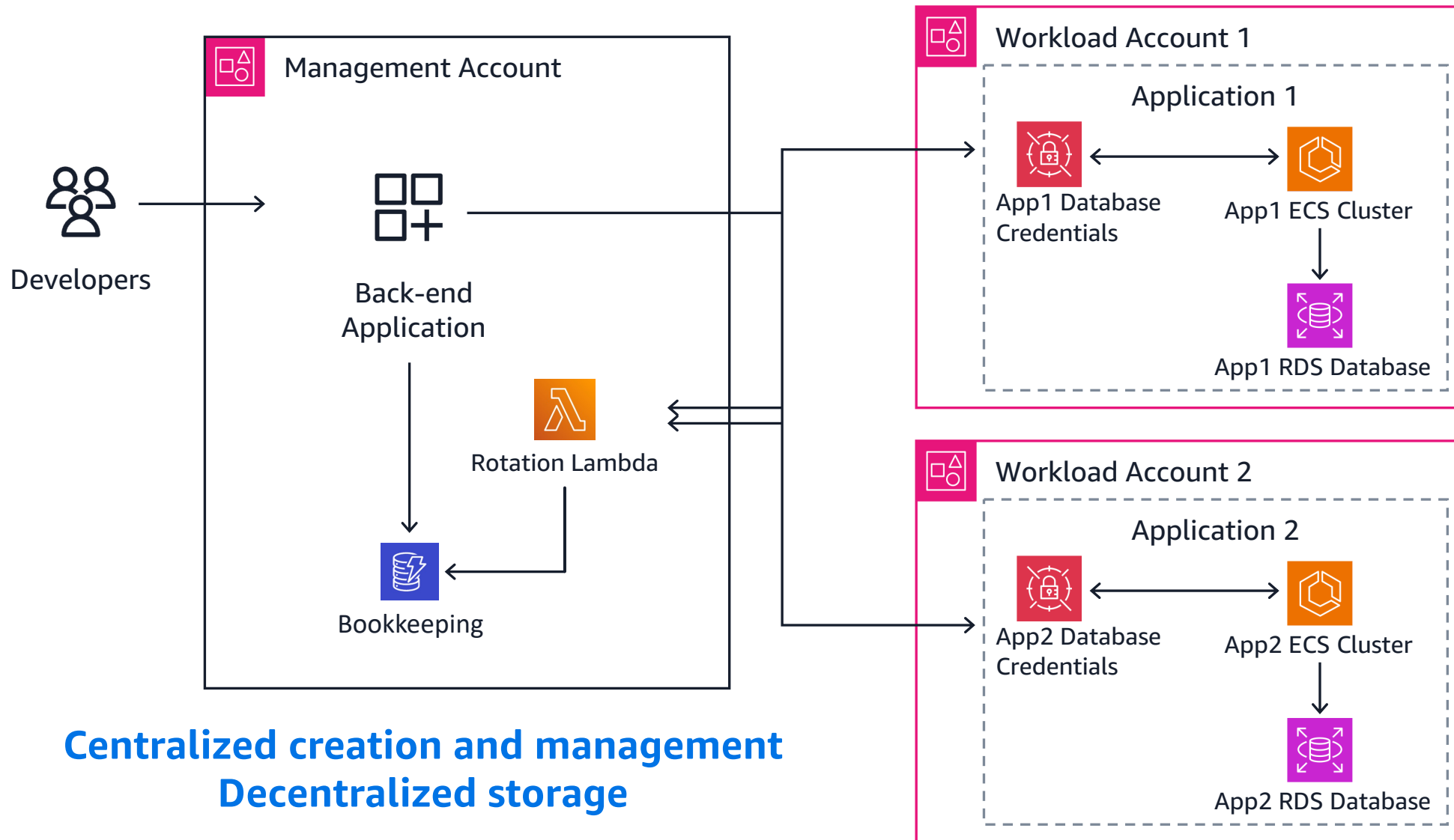
Trade-offs:

- Security/Compliance teams have limited visibility and controls
- Logs are separated into different accounts

Bottom Line:

- Gives application teams better control over least privilege permissions but requires log aggregation for visibility

Combining multiple approaches (example)



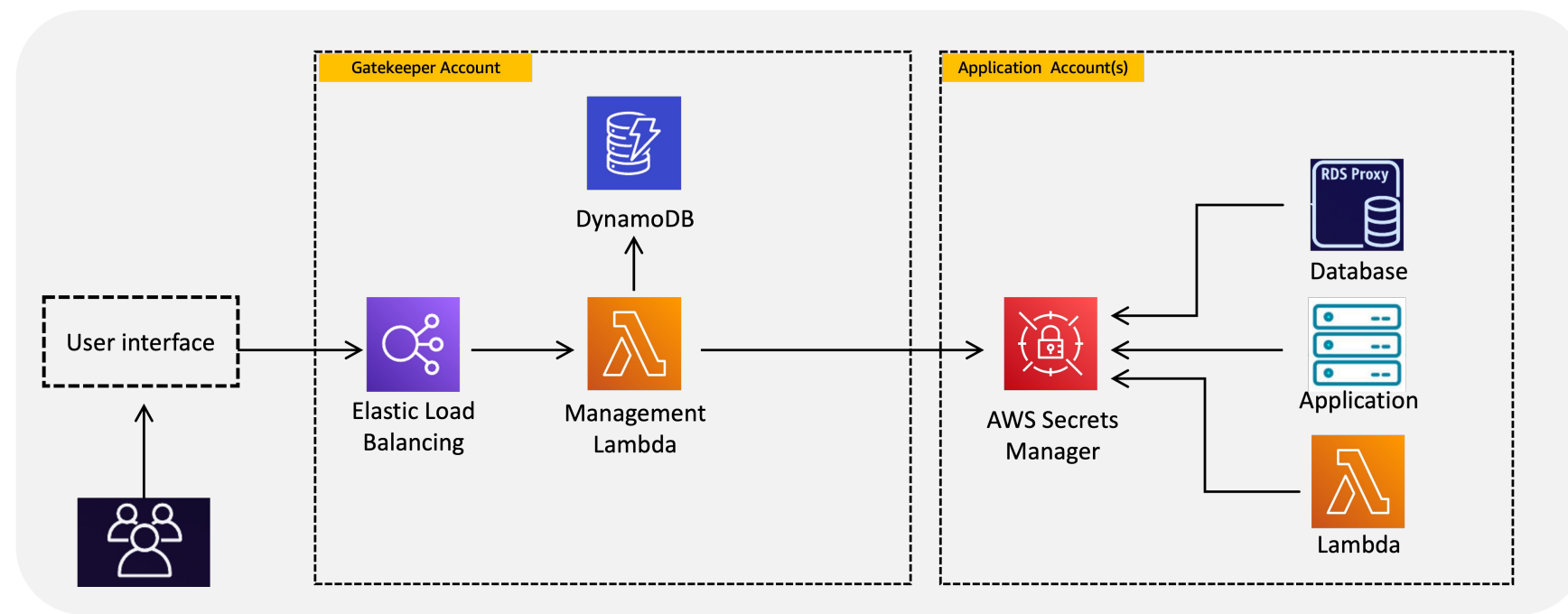
Capital One's migration to AWS Secrets Manager



Capital One's AWS
re:Inforce session

Business requirements

- Reduce costs by 90%
- Thousands of AWS accounts and business applications
- Millions of secrets: API Keys, username/password, SSH Keys, etc.
- Secrets stores in AWS Secrets Manager for RDS Proxy, ECS, etc.
- Security and compliance are top priorities
- Secrets rotation and dormant secret purging
- Cross-region failover within minutes



centralized management approach while maintaining federated (decentralized) storage for their secrets



Capital One's migration to AWS Secrets Manager



Capital One's AWS
re:Inforce session

Technical requirements

Security

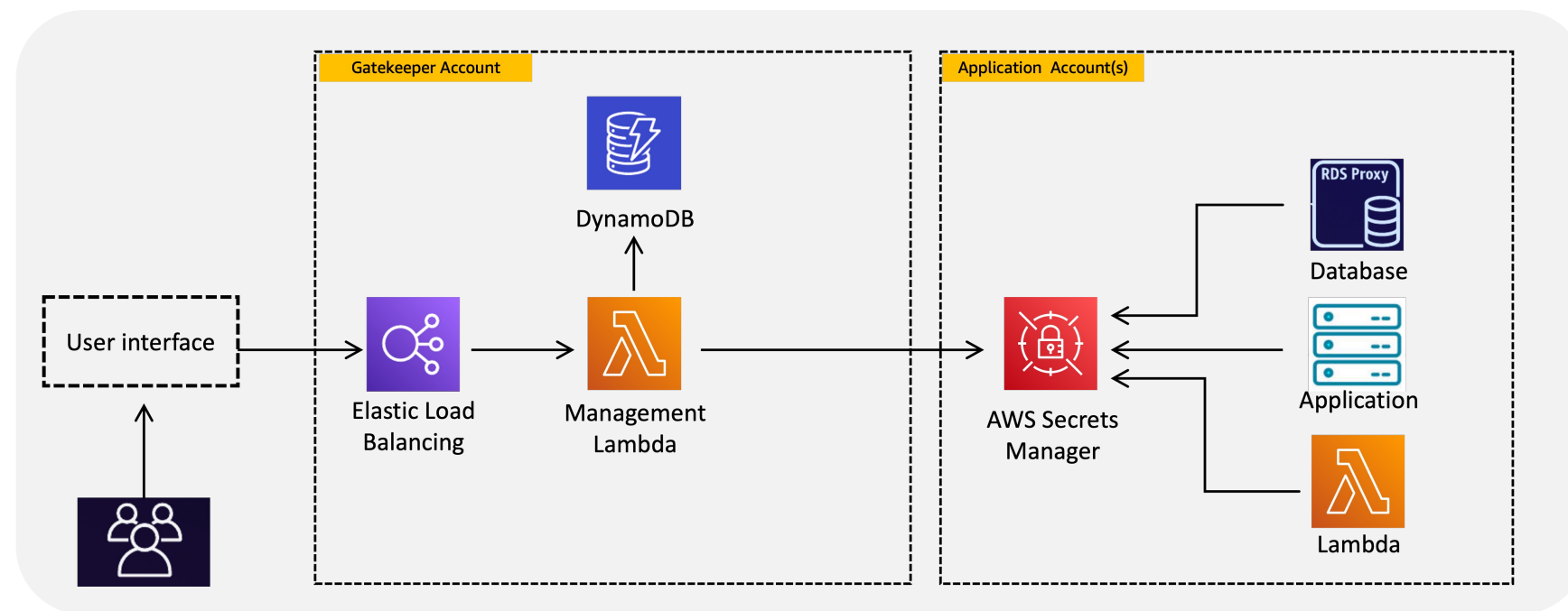
- Centrally managed through secrets gateway and federated storage
- Multi permission access control

Rotation

- Rotating the secrets in a central account
- Bring your own Lambda: application team can implement their own rotation

Resiliency

- Multi-region replication, in 4 regions (2 in US, 2 in EU)
- Active/active deployment, using Route53



centralized management approach while maintaining
federated (decentralized) storage for their secrets



“ Capital One utilizes the robust features of AWS Secrets Manager to deploy a centralized solution for managing secrets, and to meet compliance requirements. Capital One is able to deploy a solution that manages **millions of secrets** spread across thousands of AWS accounts by leveraging key capabilities such as Service Control Policies, Resource Policy, Automatic Rotation, and Attribute Based Access Controls.

Chris Schultz

Senior Director, Enterprise IAM - Authorization - Capital One



AWS Secrets Manager customers



Fannie Mae



mercado
libre



ACQUID

BREAK!
Please take survey



Secret Lifecycle Management

Secret Lifecycle Management

- Secret Classification
- Access Management
- Secure Storage
- Rotation
- Observability and Traceability
- Incident Response
- Disaster Recovery and High Availability



Secrets in AWS: offload to purpose-built services

[How to choose the right AWS service for managing secrets and configurations](#)

AWS credentials	-----	AWS Identity & Access Management (IAM)
Encryption keys	-----	AWS Key Management Service (KMS) and CloudHSM
SSH keys	-----	Amazon EC2 Instance Connect
Private Keys + Certificates	-----	AWS Certificate Manager (ACM)
Non-sensitive key-value pairs	-----	AWS Systems Manager Parameter Store



Secret Lifecycle Management

- Secret Classification
- Access Management
- Secure Storage
- Rotation
- Observability and Traceability
- Incident Response
- Disaster Recovery and High Availability

Cross-Account Access

Method 1: Role Chaining

Account A: Secret A + Role A | **Account B:** Role B

Role B in Account B assumes Role A in Account A through a trust relationship, gaining temporary credentials to retrieve Secret A into Account B.

Method 2: Resource-Based Policy (Direct Access)

Account A: Secret A | **Account B:** Role B

A resource-based policy attached directly to Secret A grants Role B in Account B the ability to access the secret without assuming an intermediary role. The resource-based policy can define permissions for any supported Secrets Manager operation.

```
"Principal": {"AWS": "arn:aws:iam::AccountId:root"}
```

Example 1: Specify only the other account ID

Example 2: Specify the exact user or role in the other account

```
"Principal": [  
  {"AWS": "arn:aws:iam::AccountId:role/MyCrossAccountRole"},  
  {"AWS": "arn:aws:iam::AccountId:user/MyUserName"}  
]
```

Scaling access using tags (ABAC)

- For rapid growth environments
- Simplifies management of access control pattern
- Create 1 or a small number of policies to scale out access to N number of secrets
- Strict tagging enforcement required

```
{
  "Version": "2012-10-17",
  "Statement": [ {
    "Effect": "Allow",
    "Action": [
      "secretsmanager:CreateSecret" ],
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "aws:RequestTag/project": "${aws:PrincipalTag/project}",
        "aws:RequestTag/costcenter": "${aws:PrincipalTag/costcenter}" },
      "ForAllValues:StringEquals": {
        "aws:TagKeys": [
          "project",
          "createdBy",
          "costcenter",
          "application" ] } } } ] }
```

Permission policy to manage secrets using tags

```
"Effect": "Allow",
"Action": ["secretsmanager:TagResource"],
"Resource": "*",
"Condition": {
    "StringEquals": {
        "secretsmanager:ResourceTag/project": "${aws:PrincipalTag/project}",
    "ForAllValues:StringEquals": {
        "aws:TagKeys": [
            "project",
            "createdBy",
            "application",
            "costcenter" ]    },
    "StringEqualsIfExists": {
        "aws:RequestTag/project": ["${aws:PrincipalTag/project}"],
        "aws:RequestTag/costcenter": [ "${aws:PrincipalTag/costcenter}" ]}}},
```

Only tag resources with these tags

Tag with either of these keys

For **project**, you specify only these values

Permission policy to manage secrets using tags

```
{  
  "Effect": "Allow",  
  "Action": [ "secretsmanager:GetResourcePolicy",  
    "secretsmanager:GetSecretValue",  
    "secretsmanager:DescribeSecret",  
    "secretsmanager:RestoreSecret",  
    "secretsmanager:PutSecretValue",  
    "secretsmanager:UpdateSecretVersionStage",  
    "secretsmanager>DeleteSecret",  
    "secretsmanager:RotateSecret",  
    "secretsmanager:CancelRotateSecret",  
    "secretsmanager:ListSecretVersionIds",  
    "secretsmanager:UpdateSecret" ],  
  "Resource": "*",  
  "Condition": {  
    "StringEquals": {  
      "secretsmanager:ResourceTag/project": "${aws:PrincipalTag/project}"  
    }  
  }  
}
```

Only manage resources
with these tags

Permission policy to require attributes on new secrets

```
{
  "Version": "2012-10-17",
  "Statement": [ {
    "Effect": "Allow",
    "Action": [
      "secretsmanager:CreateSecret" ],
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "aws:RequestTag/project": "${aws:PrincipalTag/project}",
        "aws:RequestTag/costcenter": "${aws:PrincipalTag/costcenter}" },
      "ForAllValues:StringEquals": {
        "aws:TagKeys": [
          "project",
          "createdBy",
          "costcenter",
          "application" ] } } } ] }
```

Requires **project** and **costcenter** tags and must be this value

Allows project with these keys, but nothing else

Secrets Manager access control policy types

Identity-based policies	Resource policy on secret	Resource Control Policy (RCP) for secrets	Resource policy on KMS key
1 to many	Many to 1	Many to many	Many to 1
Provide access to multiple secrets for a single IAM entity (e.g., role)	Provide access to a single secret for multiple IAM entities (e.g., roles)	Set maximum available permissions for all secrets across AWS accounts in an organization	Provide access to a single KMS key for multiple IAM entities (e.g., roles)
Control permissions to API actions for resources that don't yet exist, or grant permissions to create new resources	Only controls permissions to an existing secret – the secret to which the resource policy is attached	Controls permissions to all existing secrets in accounts where RCP is attached; cannot grant permissions, only restrict	Only controls permissions to an existing key – the secret to which the resource policy is attached
Grant access to users, roles, or groups in the same account	Grant access to users or roles in other AWS accounts	Restrict access from external accounts; enforce data perimeter controls across organization	Grant access to users or roles in other AWS accounts
Required	Optional	Optional (must be enabled at organization level)	Required

Secret Lifecycle Management

- Secret Classification
- Access Management
- Secure Storage
- Rotation
- Observability and Traceability
- Incident Response
- Disaster Recovery and High Availability

Define practices for encryption

Default service key

- Unique key for each account and region
- No overhead of managing AWS KMS permissions

AWS KMS Key

- Unique compliance or security requirements
- Required for cross-account access to secrets
- Another set of access control

Choose secret type

Secret type [Info](#)

☒ Credentials for Amazon RDS database

☐ Credentials for Amazon DocumentDB database

☐ Credentials for other database

☐ Managed external secret
Secrets vended by your third-party software vendor.

Credentials [Info](#)

User name

awsadmin

Password

.....

☐ Show password

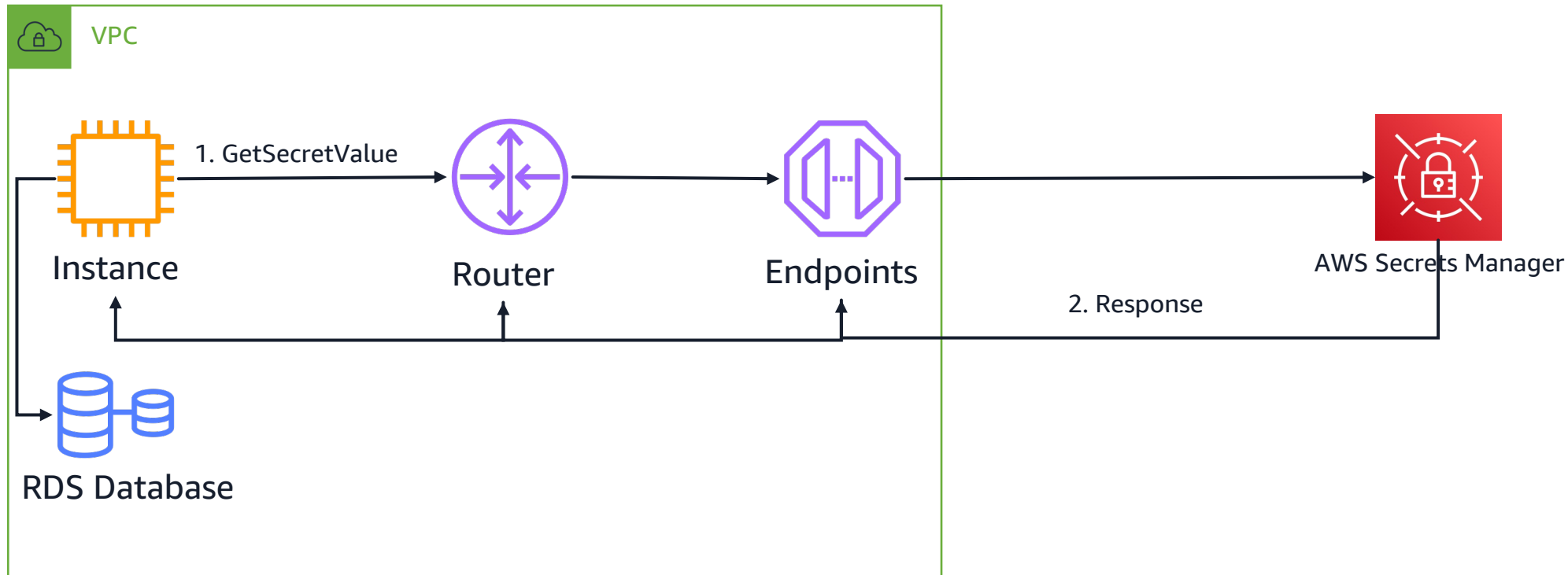
Encryption key [Info](#)

You can encrypt using the KMS key that Secrets Manager creates or a customer managed KMS key that you create.

aws/secretsmanager

[Add new key](#)

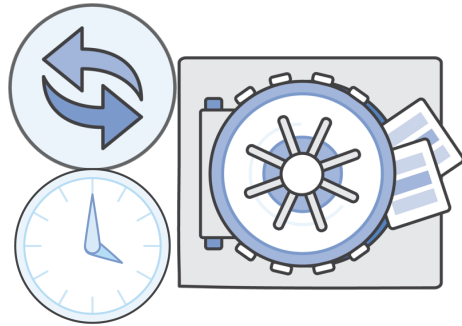
Accessing Secrets Over a Private Network



Secret Lifecycle Management

- Secret Classification
- Access Management
- Secure Storage
- Rotation
- Observability and Traceability
- Incident Response
- Disaster Recovery and High Availability

Rotate secrets safely



- **Managed secrets:** AWS offers managed rotation for **Amazon RDS, Aurora, Redshift, DocumentDB, ECS,** and **managed external secrets** (third-party credentials)
- **Extensible with AWS Lambda** for other types of secrets (custom databases)
- **Use versioning** so that applications don't break when secrets are rotated
- **Set rotation schedules** based on your security policies and compliance mandates to meet regulatory requirements
- **Transform** a long-term secret **into a short-term secret** that is rotated automatically

Fully managed database admin secrets

Overview

- You want to securely store and manage administrator credentials to your [Amazon RDS](#), [Redshift](#), [Aurora](#), [DocumentDB](#) databases with as little operational overhead as possible
- When you create or modify your database, you can allow the database service to manage the admin password in Secrets Manager, including automatic secret rotation
- Database service generates the password and stores it in Secrets Manager

Benefits

- You do not need to configure or manage AWS Lambdas for secret rotation
- Password is generated by the relevant database service (e.g., Amazon RDS) and is never visible to your administrators
- You are in full control of access permissions and encryption key configuration, like any other secret in Secrets Manager



Amazon RDS



Amazon
Redshift



Amazon
Aurora



Amazon
DocumentDB

Prebuilt secret rotation templates

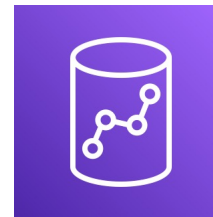
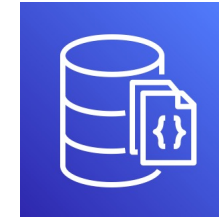
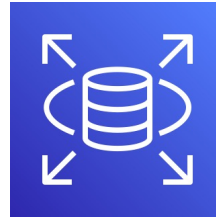
When you enable rotation for a supported secret, [Secrets Manager](#) provides a [Lambda rotation function](#) for you and populates the function automatically with the ARN in the secret



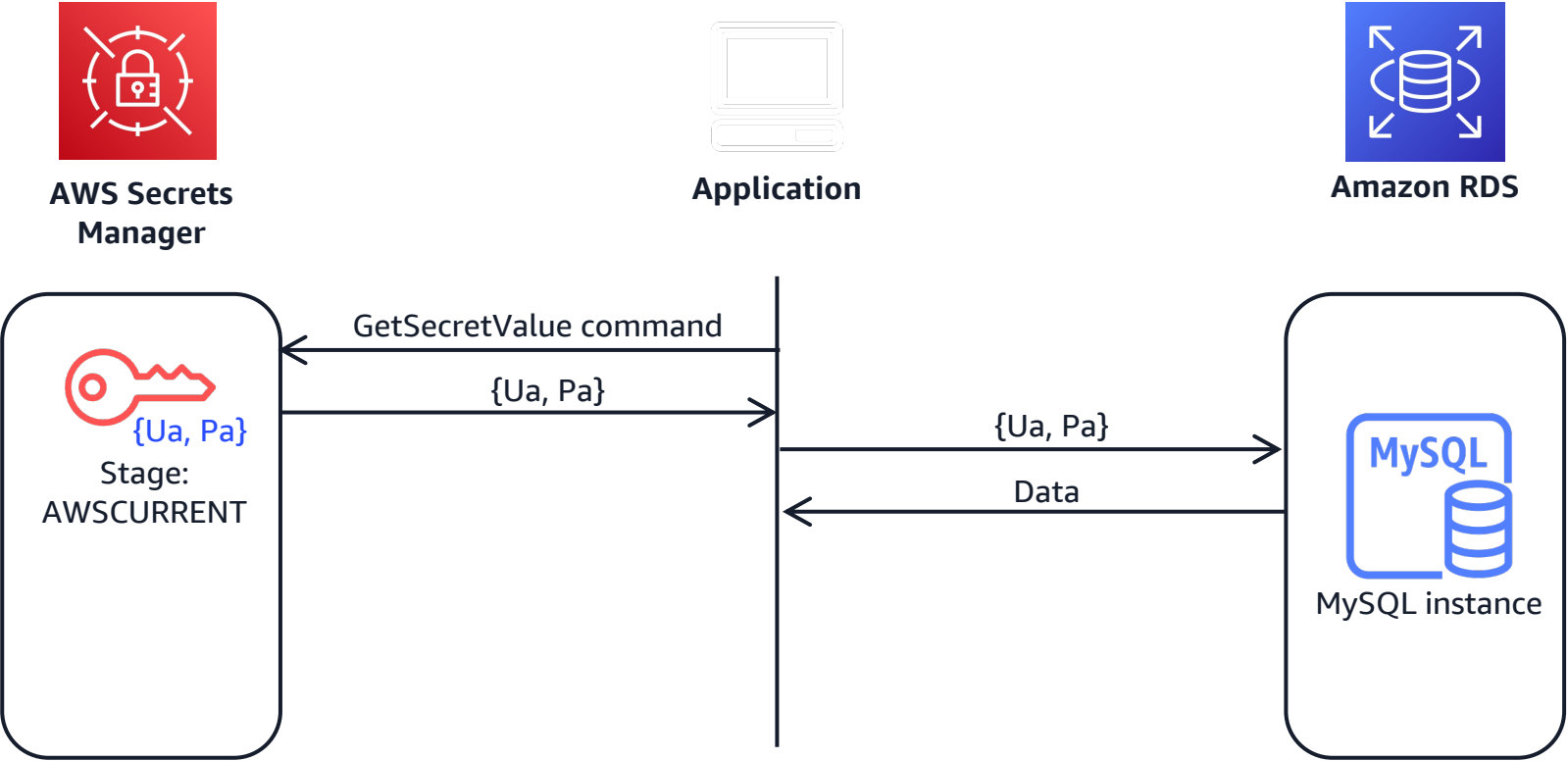
[AWS Secrets Manager Rotation Lambda Functions](#)

Supported Database Types

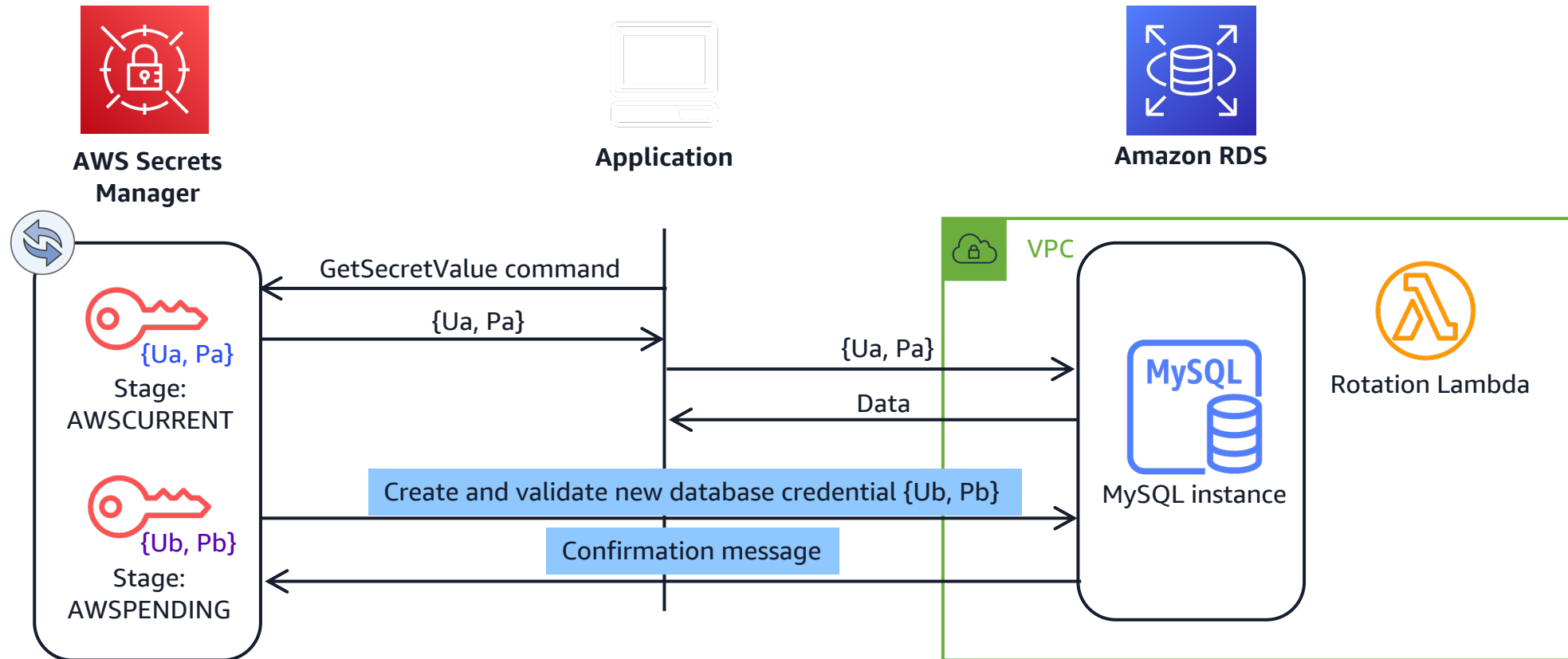
- RDS Maria DB
- RDS MySQL
- RDS Oracle
- RDS PostgreSQL
- RDS Microsoft SQLServer
- Redshift
- DocumentDB



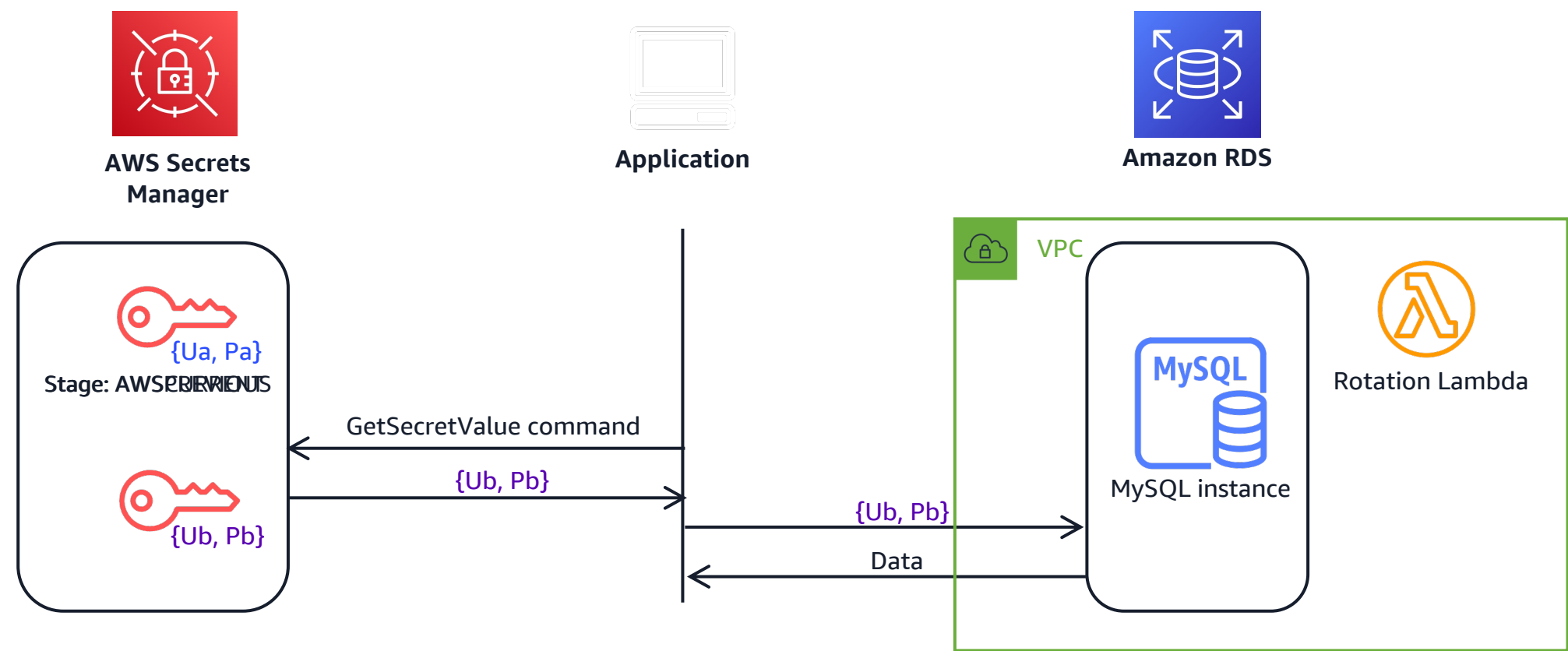
How rotation works – secret retrieval



How rotation works – secret rotation (single user)

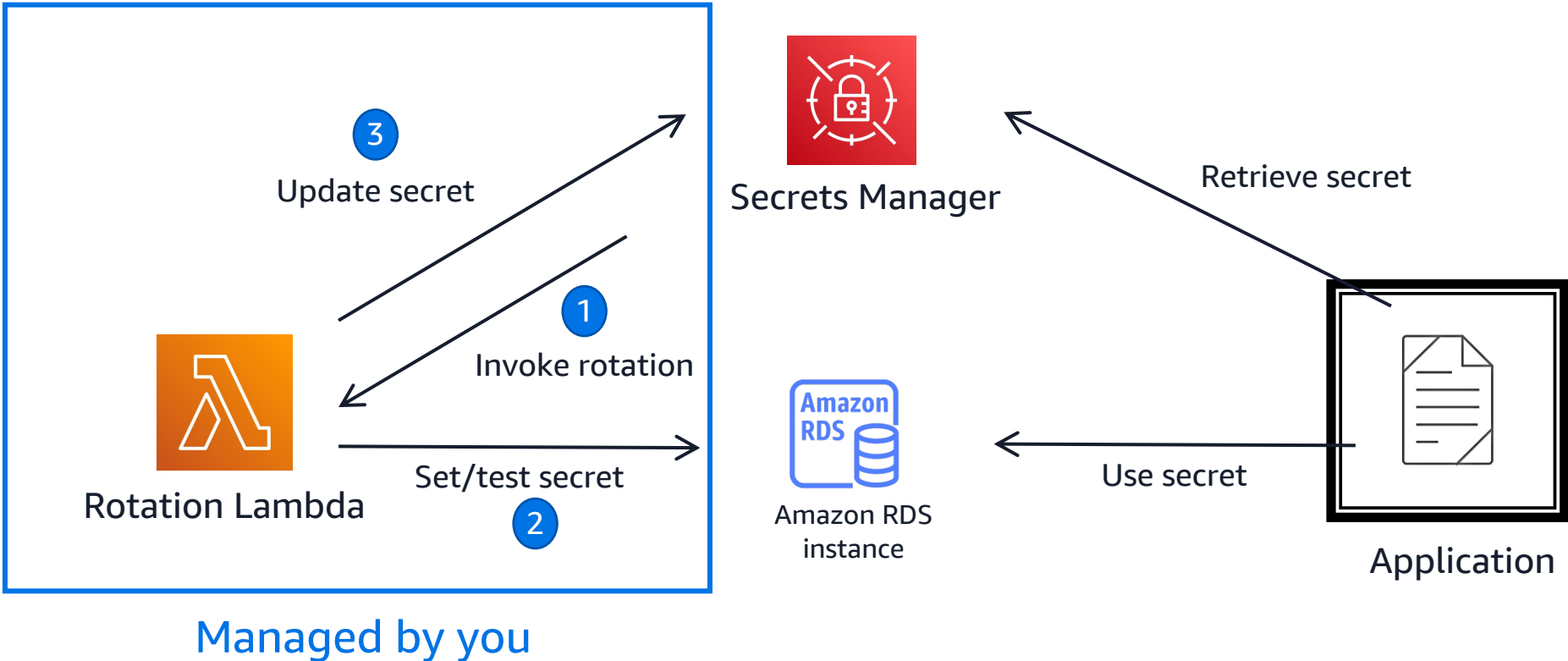


How rotation works – post rotation



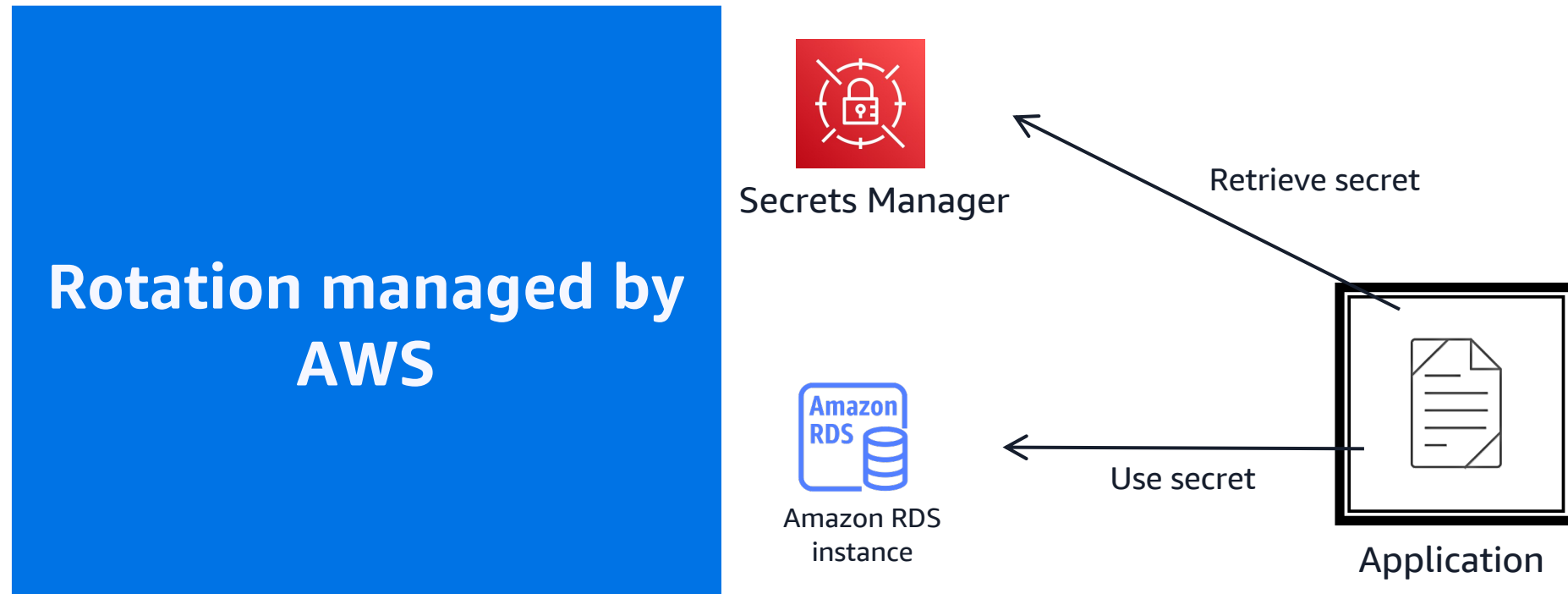
Secret rotation managed by you

For secrets that you create, you are responsible for managing rotation Lambdas



Secret rotation managed by AWS

When you enable admin password management with Secrets Manager, AWS manages rotation on your behalf



Rotation Prerequisites

Permissions

- Create an IAM policy with permissions to set up the [rotation configuration](#)
- Create a separate policy to trigger rotation
- [Lambda IAM role is used for rotation](#) and any required KMS operations

Networking

- Lambda needs to communicate with both the protected service or database and [Secrets Manager](#) endpoint
- For supported services the [CloudFormation](#) template determines public accessibility
- By default, the [Secrets Manager](#) endpoints run on the public internet

Rotation windows for secret rotation

Secrets Manager now supports scheduling secret rotations to occur within a specific time period on specific days. **You can now rotate secrets up to every four hours!** Previously, secrets could be rotated up to once a day, sometime within the last 24 hours of the rotation period.

Rotation Window Benefits

- Schedule secret rotation to occur during your company's existing change windows and better adhere to your change management processes
- Reduce or eliminate impact to your business by rotating database secrets during off-hours for your applications

Rotation Window Configuration

- Rotation windows can be created using a simple interactive wizard in the AWS console
- Rotation settings can also be specified via a cron expression in the AWS console, CLI, or using CloudFormation

Rotation must not affect application availability:

single user vs. multi-user rotation strategies

Single user rotation strategy

- Rotates credentials for one user
- User must have permission to update their own password
- Rotations can be scheduled to align with maintenance windows
- Low risk of access denial during rotation (use a retry strategy!)

Alternating user rotation strategy (multi-user)

- First rotation: Lambda function clones the original user
- Subsequent rotations alternate between the two users
- Requires 2 or more sets of valid credentials for the service owning the secret
- Rotation function may require a “superuser” that has permissions to clone a user
- Best used for high availability or unique permissions models

Secret Lifecycle Management

- Secret Classification
- Access Management
- Secure Storage
- Rotation
- Observability and Traceability
- Incident Response
- Disaster Recovery and High Availability

Audit and monitor secrets

Supporting Services

Visibility & Compliance

- **IAM Access Analyzer:** Quick glance
- **AWS CloudTrail Logs:** Auditable records
- **AWS Config:** Track guidelines and activity

Alerting & Automation

- **Amazon EventBridge:** Monitor use
- **Amazon CloudWatch Alarms:** Alerting and thresholds

Tips

- Monitor attempts to retrieve secrets that are **scheduled for deletion**
- Monitor high-value secrets
- CloudTrail records all Secrets Manager API calls; **expect an increase in the size of your trails**
- Set CloudWatch Alarms on:
 - `GetSecretValue` API call spikes (via CloudTrail metric filters) to detect credential harvesting attempts
 - `RotationFailed` CloudWatch metrics to catch broken rotation before secrets expire

IAM Access Analyzer

AWS IAM Access Analyzer is a free service that helps you **identify the resources** in your organization and accounts, such as Secrets Manager secrets, **that are shared with an external entity**.

This lets you **identify unintended access** to your resources and data, which is a security risk.

IAM Access Advisor helps you audit access and **remove unnecessary permissions**. Use it frequently to scope appropriate permissions.



CloudTrail: Compliance Monitoring, Alerting, and Auditability

- Determine what action was performed with your secret
- Determine who accessed the secret and when
- See associated metadata with the event
- Various integration paths for long term log archival

CloudTrail > Event history > GetSecretValue

GetSecretValue [Info](#)

Details [Info](#)

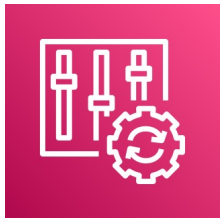
Event time January 17, 2023, 15:35:31 (UTC-08:00)	AWS access key ASIAZEG5UIFIMF6SWOZN	AWS region us-west-2
User name bkolasin-lsengard	Source IP address 205.251.233.183	Error code -
Event name GetSecretValue	Event ID 31d34980-d3ec-4574-b551-e492af8fb4f5	Read-only true
Event source secretsmanager.amazonaws.com	Request ID 7899ece2-5c06-41cf-8501-84184684ecfb	

Resources referenced (1) [Info](#)

Resource type	Resource name
AWS::SecretsManager::Secret	arn:aws:secretsmanager:us-west-2:627530023248:secret:demo/database-1-UJL

Audit and monitor using AWS Config

- Review changes in configurations and relationships between AWS resources
- Track [detailed resource configuration histories](#)
- Determine your [overall compliance](#) with configurations specified in your internal guidelines



AWS Config

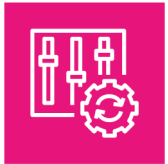
Managed Rules

- Secretsmanager-rotation-enabled-check
- Secretsmanager-scheduled-rotation-success-check
- Secretsmanager-using-cmk
- Secretsmanager-secret-unused
- Secretsmanager-secret-periodic-rotation

Secret Lifecycle Management

- Secret Classification
- Access Management
- Secure Storage
- Rotation
- Observability and Traceability
- Incident Response
- Disaster Recovery and High Availability

Monitor secrets for compliance and detect threats with **AI-powered AWS integrations**



AWS Config



AWS Security Hub



Amazon GuardDuty



Amazon Inspector



Kiro

AWS Config

Monitor secrets for compliance and identify secrets that don't conform to policies

AWS Security Hub

Evaluate secrets rotation, encryption, and access controls against PCI DSS 4.0.1

Amazon GuardDuty

Detect suspicious access patterns to secrets with AI-powered threat detection; automatically route findings to Security Hub

Amazon Inspector

Detect exposed secrets in container images and application code

Kiro

AI-powered IDE plugin detects hardcoded secrets in real-time

Secret Lifecycle Management

- Secret Classification
- Access Management
- Secure Storage
- Rotation
- Observability and Traceability
- Incident Response
- Disaster Recovery and High Availability

Multi-region replication overview

Multi-Region Replication allows customers to create regional read replicas for secrets, and select the regions where *replica secrets* should be created. Any update made to the primary secret, such as a secret value updated through automatic rotation, will be automatically propagated to the replica secrets.

Secret Replication Use Cases

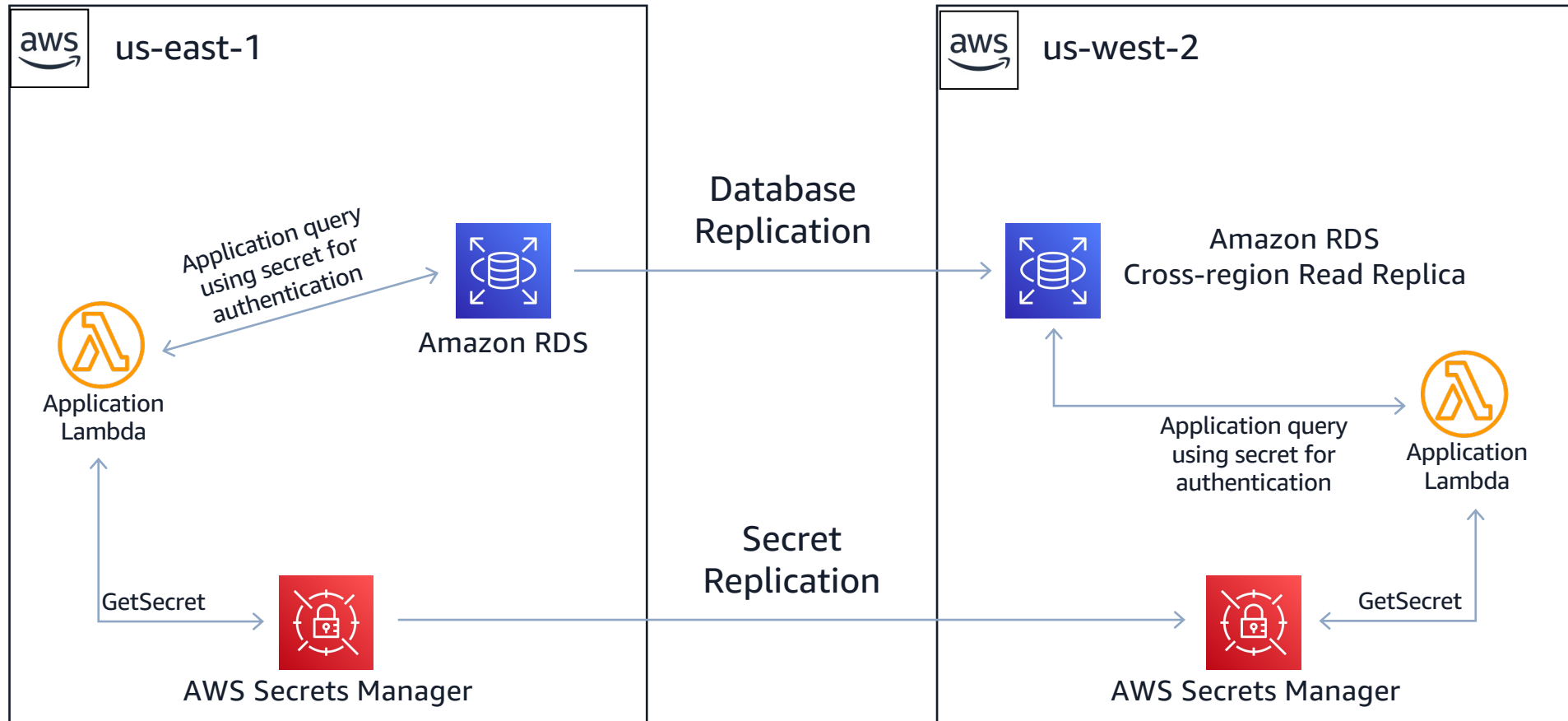
- Support application *high availability*
- Retain access to secrets during *disaster recovery* scenarios (e.g. Region outage)

Secret Replication Considerations

- Replicates secret and associated metadata to one or more regions
- Secrets have a common name across regions
- Secret rotation in primary region propagates to *replica secrets* in all regions (where enabled)

Multi-Region Replication Example: Disaster Recovery

Creating **replica secrets** in multiple AWS regions allows customers to plan for potential disaster recovery scenarios where failover to a new region is necessary. In the below example, a replica secret (replicated from us-east-1) is used by a serverless application in us-west-2 to access an RDS Cross-Region Read Replica.



AWS Secrets Manager Migration Guidance

Migration journey overview

Phase 1: Discovery & Classification



[Migrate your secrets,
Part 1: Discovery and
design blog](#)

- **Inventory your secrets** across applications and databases
- **Evaluate each secret:** Retire, Retain, or Relocate
- **Capture metadata** for tag-based access control

Phase 2: Design



[Exploring common
centralized and
decentralized approaches
blog](#)

- **Choose centralized or decentralized** approaches for four aspects: creation, storage, rotation, and monitoring
- **Define tag-based access controls** (ABAC) using naming and tagging conventions
- **Configure VPC endpoints** for private connectivity
- **Plan rotation strategy** per secret type

Phase 3: Implementation



[Migrate your secrets,
Part 2: Implementation
blog](#)

- **Choose migration path:** migrate application first then update code, OR update on-premises app to use Secrets Manager (using IAMRA)
- **Verify cutover checklist:** API access, correct tags, IAM permissions, documented rollback plan
- **Enable** rotation, monitoring, governance controls, and threat detection (Amazon GuardDuty)

Accelerate Your Secrets Migration with AWS Transform



Automate code-level migration from HashiCorp Vault to AWS Secrets Manager

AWS Transform is the first agentic AI service for large-scale migration and modernization.

The Secrets Management Migration Plugin:

- Scans application code for HashiCorp Vault API calls and dependencies
- Automatically refactors code to use AWS Secrets Manager instead
- Handles client library replacement, authentication flow updates, and API translation
- Reduces manual code refactoring effort from weeks to minutes



Scan to access the
GitHub repository

Get started: github.com/aws-samples/sample-secrets-management-code-migration-with-aws-transform-custom

Knowledge Check



Trivia Time

Join Interactive Quiz:

1. Navigate to Trivia on Skills Builder

- QR Code
- <https://trivia.skillbuilder.aws/session/join/918707>
- Navigate to **trivia.skillbuilder.aws** and enter code **918707**

2. Choose username

3. Press Join

4. Wait for Quiz to Start



AWS Secrets Manager Workshop:

Rise to the Challenge of Secrets Management in the Cloud

<https://catalog.workshops.aws/secrets-management-challenge/en-US>

Workshop outline

Module 1: Control Access to your secrets

Module 2: Use In-Memory secret caching leveraging AWS Secrets Manager Agent

Module 3: Secrets Lifecycle - rotation challenge

Module 4: Cross-region secret fetching

Getting started with Workshop Studio



You have access to an AWS account with everything needed to complete this workshop.



The AWS account is only available for the duration of this session. **You will lose access to the account once the session is complete.**



The workshop environment is deployed to a specific AWS Region. Make sure that you are working in this Region; other Regions are blocked.



Review the terms and conditions of the event. **Do not upload any personal or confidential information to the account.**



Accessing the workshop environment for this session

01

Sign in using your preferred method – <https://join.workshops.aws>

02

Enter the event access code – **5140-082c42-86**

03

Review terms and conditions
join event

04

Select **Get Started** to begin the workshop

05

Access AWS accounts – access the AWS Management Console

Recap

Module 1: Key Concepts: Controlling Access to Secrets



How access control works in Secrets Manager

Two layers of access control:

IAM Policies define WHO can call Secrets Manager APIs (identity-based)

Resource Policies define WHO can access a specific secret (resource-based)

Both policies are evaluated together. Access is denied unless explicitly allowed by both.

Best Practices for this module:

Grant only `secretsmanager:GetSecretValue` (not `secretsmanager:*`)

- Use conditions (`aws:SourceVpc`, `aws:PrincipalTag`) to restrict access further
- Resource policies enable cross-account access without role chaining

Module 2: Key Concepts: Secrets Manager Agent



What is the Secrets Manager Agent?

A lightweight, language-agnostic local HTTP client that caches secrets in memory.

How it works:

- Your application calls localhost:2773 to retrieve secrets
- Cache HIT: Secret returned instantly from memory (no API call)
- Cache MISS: Agent calls Secrets Manager API, caches result, returns to app

Benefits:

- Reduced latency (microseconds vs. milliseconds for API calls)
- Reduced cost (fewer GetSecretValue API calls)
- Works with any language/framework that can make HTTP requests
- Configurable TTL controls how long secrets stay cached

Module 3: Key Concepts: Secret Rotation



Why rotate secrets and how it works

Why rotate?

- Limits blast radius of compromised credentials
- Meets compliance requirements (PCI DSS, SOC2, HIPAA)
- Reduces risk window from months/years to hours/days
- Automated rotation removes human error from the process

Two rotation strategies:

Single-user rotation:

One credential rotated in place. Brief window where old cred is invalid before new one propagates.

Alternating-user rotation:

Two credentials swap active/standby. Zero downtime because old cred remains valid until next rotation.

Rotation is handled by a Lambda function (or managed rotation for supported databases).

Module 4: Key Concepts: Cross-Region Secret Replication



Multi-Region replication for resilience

What is it:

Secrets Manager can automatically replicate secrets to multiple AWS Regions, creating read replicas that stay in sync with the primary secret.

Why it matters:

- Disaster recovery: applications fail over to another Region and still access secrets
- Low-latency reads: applications retrieve secrets from their local Region
- Simplified multi-Region deployments: one secret, multiple Regions, single rotation config

How it works in this module:

- You will replicate a secret from us-east-1 to eu-west-3
- Update the Secrets Manager Agent config to fetch from the replica Region
- Observe that your application continues to function with the replicated secret

AWS Secrets Manager Best Practices

<https://docs.aws.amazon.com/secretsmanager/latest/userguide/best-practices.html>



Store: Store all sensitive credentials in Secrets Manager (never in code, config files, or environment variables)

Detect: Find unprotected secrets in code with Kiro

Encrypt: Use AWS managed key (free) for most cases; use customer managed KMS key for: cross-account access, granular key policies, compliance requirements, or independent key lifecycle control

Cache: Use client-side caching (Secrets Manager Agent, SDK libraries, Lambda extension) to reduce API calls and cost

Rotate: Enable automatic rotation (as often as every 4 hours); use single-user or alternating-user strategy

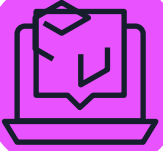
Access: Apply least-privilege with IAM policies, resource policies, and ABAC; use BlockPublicPolicy

Replicate: Replicate secrets across Regions for resilience and disaster recovery

Monitor: Enable CloudTrail logging, CloudWatch metrics, Config rules, and GuardDuty threat detection

✗ Don't create a new KMS key for every secret. Secrets Manager uses envelope encryption: each secret gets its own unique data key, then encrypted by the KMS key. Sharing one KMS key across many secrets does NOT weaken security. Each KMS key costs \$1.00/month. With 1,000 secrets, individual keys = \$1,000/month wasted for zero additional protection. Use one KMS key for many secrets (or the free AWS managed key for most cases).

Learn in-demand AWS Cloud skills



AWS Skill Builder

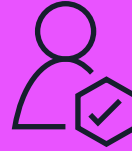
Access **500+ free** digital courses and Learning Plans

Explore resources with a variety of skill levels and **16+** languages to meet your learning needs

Deepen your skills with digital learning on demand



Train now



AWS Certifications

Earn an industry-recognized credential

Receive Foundational, Associate, Professional, and Specialty certifications

Join the **AWS Certified community** and get exclusive benefits



Access **new** exam guides

Thank You



Please complete the Survey